

QCon
全球软件开发大会

AI 搜索场景下的生成式 UI 工程实践

邓小红

百度 资深工程师



极客邦科技 2026 年会议规划

促进软件开发及相关领域知识与创新的传播



参会咨询

🕒 6月 📍 上海

👥 1000人

AiCon

全球人工智能开发与应用大会

会议时间：6月26-27日

- AI Infra 系统工程
- 多 Agent 协作与实践
- 多模态融合
- 模型训练与推理创新
- 数据平台与特征服务

🕒 8月 📍 深圳

👥 1000人

AiCon

全球人工智能开发与应用大会

会议时间：8月21-22日

- Agentic AI
- 轻量化与高效推理
- 多模态应用
- AI + IoT 场景实践
- AI 工业化落地

🕒 10月 📍 上海

👥 1200人

QCon

全球软件开发大会

会议时间：10月22-24日

- AI Agent
- Vibe Coding
- 智能可观测
- 推理基建
- 模型攻防
- AI x 创造力

🕒 12月 📍 北京

👥 1000人

AiCon

全球人工智能开发与应用大会

会议时间：12月18-19日

- 大模型架构创新
- 多模态 AI 产业融合
- 具身智能
- AI for Science
- 大模型安全

目录

01

引言

02

AI 搜索应用中的 UI 工程实践

03

生成式 UI 自动化产线的工程探索

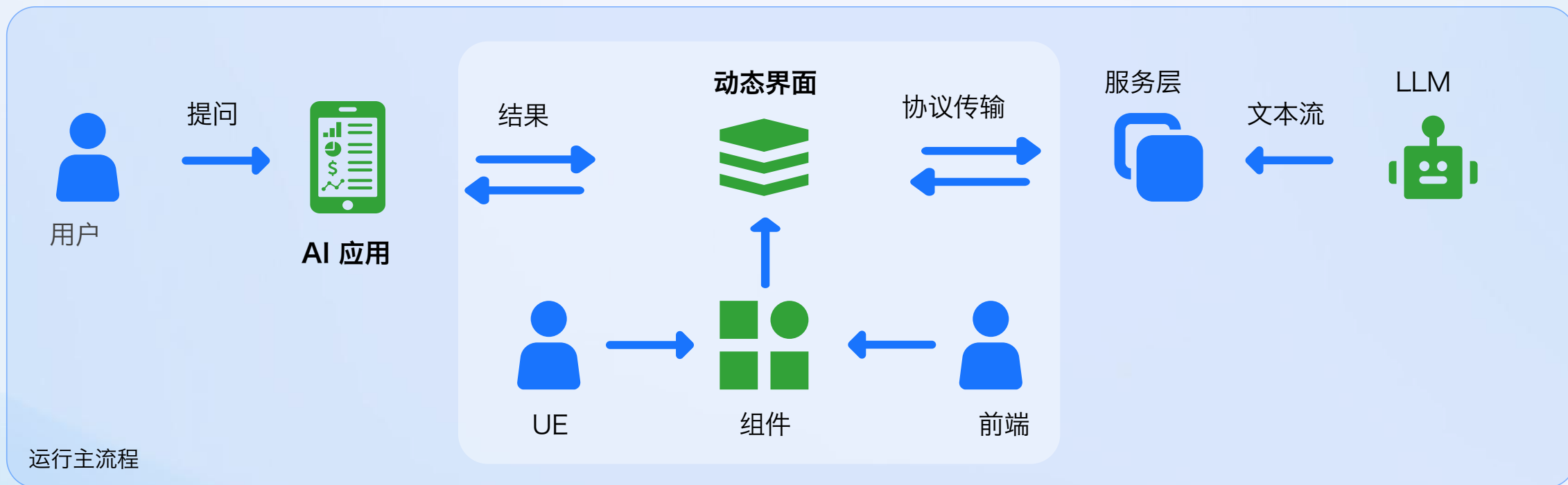
04

总结

01 引言

● 引言 | 交互范式的变化

近年来，大语言模型能力爆发式增长，涌现了大量以 AI 对话、生成式、智能体等形态为主的 AI 应用。



用户以自然语言表达“想要什么”，UI 逐步成为模型生成的重要部分。

如何用协议描述动态 UI？如何规模化生成动态 UI？

02

AI 搜索应用中的 UI 工程实践

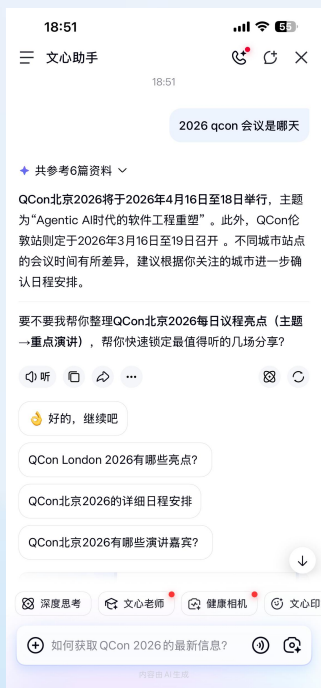
02

AI 搜索应用中的 UI 工程实践

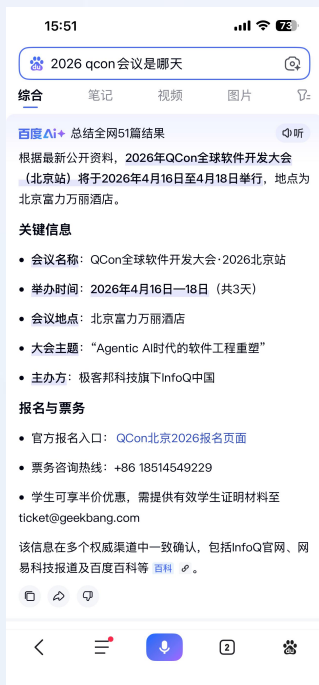
Markdown 扩展协议

AI 搜索应用中的 UI 工程实践 | 标准 Markdown 下的需求

常规富文本

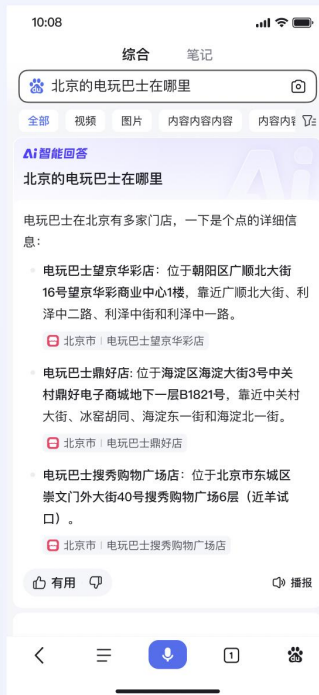


加点颜色 加点按钮



微调标准 Markdown

加点结构化



结构化越来越多

加点联动



交互越来越复杂



需求越来越复杂

加加加

标准 Markdown 即可满足



标准之下 要扩展的内容越来越多 ...

AI 搜索应用中的 UI 工程实践 | 标准 Markdown 下面临的问题



标准化问题：功能样式越来越复杂，模型该怎么描述

该怎么抽象协议？



规范化问题：功能交互越来越复杂，组件怎么建设

该怎么抽象组件？



效率问题：原有业务资产怎么复用到新业务中

该怎么复用？



速度压力：功能数量越来越多，同时对性能有要求

该怎么保证性能？

AI 搜索应用中的 UI 工程实践 | Markdown 扩展协议

Markdown 扩展协议：一套面向 LLM（大语言模型）的统一 Markdown 扩展规范，标准化 UI 描述与数据传输，确保可移植、可扩展且安全。

配套协议解析 - Marklang



Markdown 扩展协议 | 该怎么抽象协议?

问题

- Markdown 表达能力有限
- 无法承载复杂 UI / 组件
- 扩展方式碎片化
- LLM 输出不稳定

核心抽象：扩展协议

选基准规范

基于 GFM 规范。

在 CommonMark 基础上加入表格、列表等扩展特性

定扩展形态

Directive 扩展

name + content + attributes

分层语义

:内联 | ::块级 | :::容器

定扩展规范

: 为修饰符, ml 为前缀

内联指令: :ml-name[content]{key=val}

块级指令: ::ml-name[content]{key=val}

容器指令: :::ml-name[content]{ ... } :::

数据容器: :::ml-data{ ... } :::

抽象原则

- 不破坏标准 Markdown
- 统一语法形态
- 易解析
- LLM 语义友好
- 可扩展

设计原则

- 标准化扩展
- 声明式定义
- 安全隔离
- 数据解耦

Markdown 扩展协议 | 协议示例

基础扩展增强

内联扩展

```
## ==高亮标题==  
==**高亮加粗内容**==  
~一个波浪不是删除~ ~~两个波浪是删除~~
```

```
**加粗 :ml-copy[加粗复制]** :ml-tts[播放]{src="http://source.mp3"}  
[带文本icon的链接](http://www.baidu.com){type="text"} [带视频icon的链接](http://www.baidu.com)  
{type="video"} [带笔记icon的链接](http://www.baidu.com){type="note"}  
这是 <a href="https://www.baidu.com">一个不会被解析的 A 标签</a>
```

```
一个地图扩展(点击可展开)  
:ml-poi{data="poi-1"}
```

```
:::ml-data{name=poi-1}  
```json  
{
 "mapImage": "https://xxx",
 "ratio": 3,
 "linkInfo": {
 "href": "https://xx",
 "marker": true,
 "area": "北京市",
 "address": "西城区某立交桥",
 "folded": true
 }
}
```  
:::
```

```
一个店铺扩展  
::ml-shop-address{data="shop-address-1"}
```

```
:::ml-data{name=shop-address-1}  
```json  
{
 "thumbnail": "https://poi-pic-gz.cdn.bcebos.com/001/161.jpeg",
 "title": "火锅西单店",
 "distance": "20km",
 "tags": [{
 "text": "403人访问过"
 }],
 "linkInfo": {
 "href": "https://xx",
 "poi": {
 "name": "望京店",
 "area": "北京市",
 "address": "北京市西城区西单北大街",
 "folded": false
 }
 }
}
```  
:::
```

数据容器扩展

高亮标题

高亮加粗内容

~一个波浪不是删除~ 两个波浪是删除

加粗 加粗复制  播放 

 带文本icon的链接  带视频icon的链接  带笔记icon的链接

这是 一个不会被解析的 A 标签

一个地图扩展(点击可展开)

 北京市 | 西城区某立交桥

一个店铺扩展



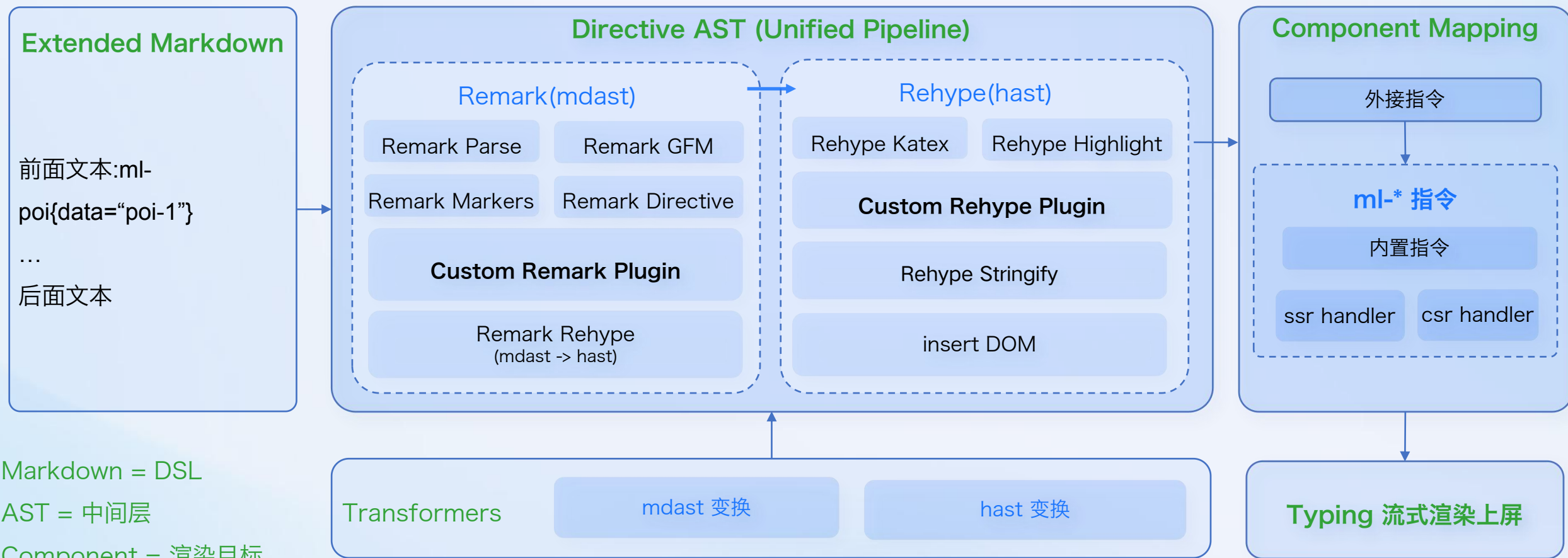
火锅西单店

20km | 403人访问过 北京市西城区西...



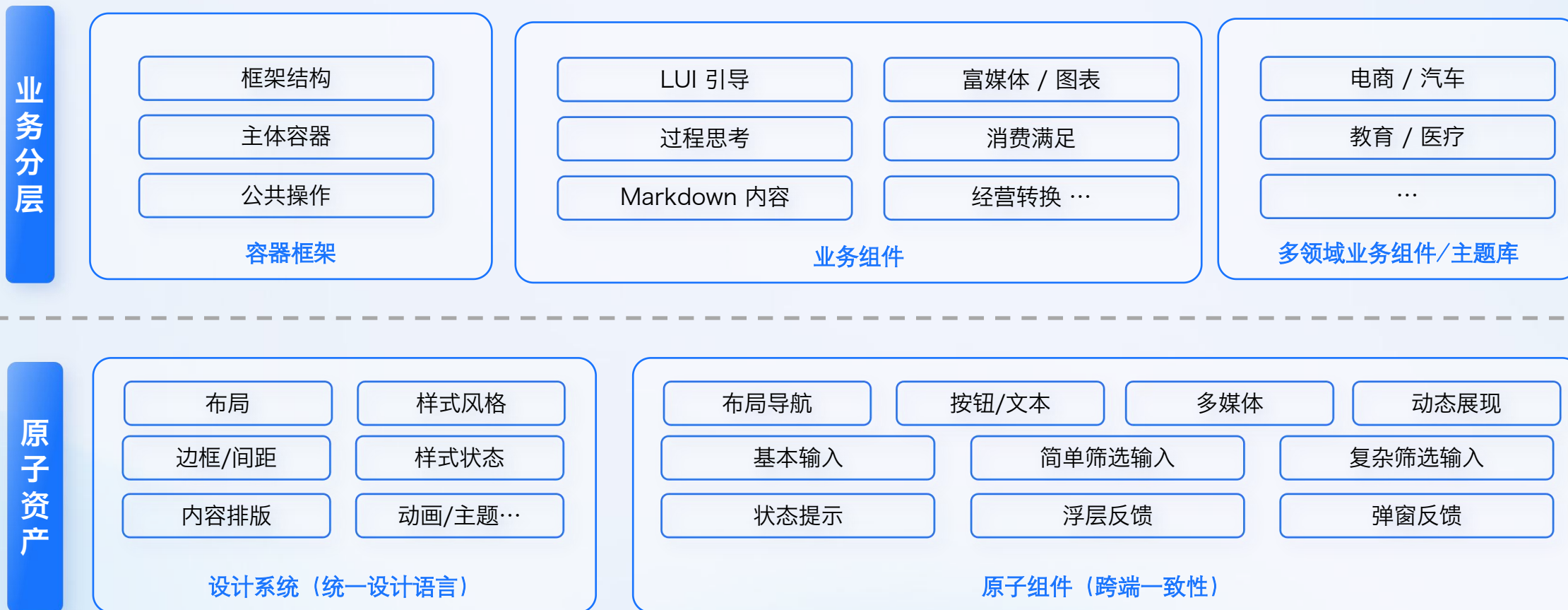
Markdown 扩展协议 | 怎么抽象组件&怎么复用?

讨论怎么抽象组件和复用之前, 先看从 Markdown 扩展协议到渲染呈现是怎么实现的?



Markdown 扩展协议 | 怎么抽象组件&怎么复用?

设计统一、语义统一、跨端(pc/mobile)、多渲染(csr/ssr)、工具化输出



Markdown 扩展协议 | 怎么保证性能?

问题：扩展协议(指令/插件/打字机)带来丰富能力的同时，叠加了多层运行开销

解决：按需加载 · 缓存复用 · 增量更新 · 渲染隔离

1 插件体积过大 & 加载时机



按需加载 + 缓存复用

- Loaded -> Loading -> dynamic import
- promise 复用防重复
- cachedInstance 无变化直接复用，不重建

2 流式渲染开销



分段渲染

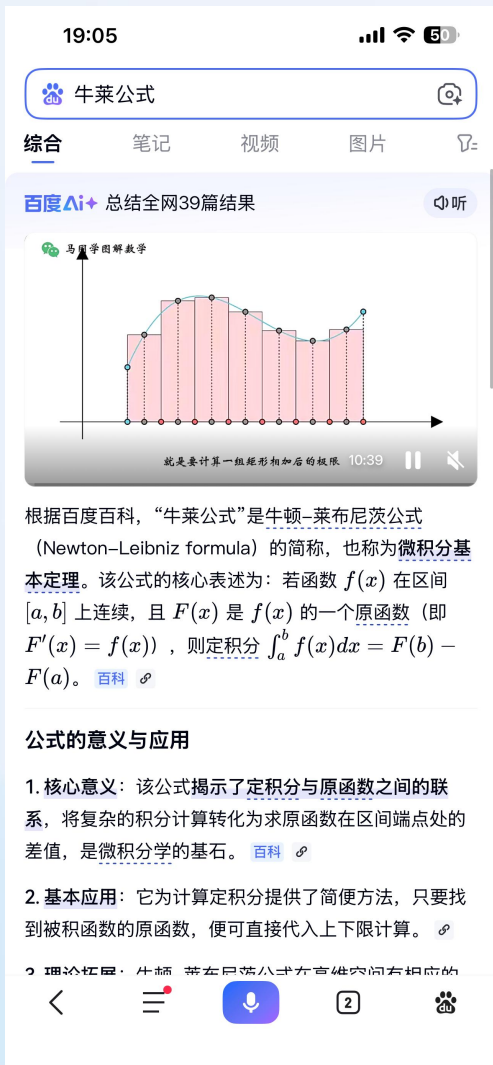
- SSR hydrate 支持 SSR 模式，复用服务端 DOM
- 针对不同模式，隔离重建、离屏渲染

3 Pipeline 重建 & AST 操作频繁



Pipeline 缓存 & 批量操作

Markdown 扩展协议 | 协议应用



02

AI 搜索应用中的 UI 工程实践

JSON 结构化协议

AI 搜索应用中的 UI 工程实践 | JSON 结构化协议的背景



聚合不同行业信息但不同行业数据结构、效果定制不同，如何保证在同一运行环境内复用组件且分发规范？

AI 搜索应用中的 UI 工程实践 | JSON 结构化协议

协议构成:

UI Schema: 描述 UI 的结构

- 节点类型: 容器节点、内容节点
- 节点描述: 节点属性、子节点、逻辑控制(条件&循环)、交互行为。内部允许动态插值。

Data Model: 描述渲染所需的静态值及上下文变量值。可分段传输。

```
interface UISchema {
  ui: UI;
  dataExtends?: dataExtends;
}

interface DataModel {
  data?: Data;
}

interface UI {
  // 节点类型
  type: string;

  // 节点属性 含插值&作用域
  props?: PropsConfig;

  // 条件
  if?: string;

  // 循环
  for?: string;

  // 子节点
  children?: UI[];

  // 交互行为、含插值&作用域
  events?: {
    [key: string]: ActionConfig[];
  };
}
```

```
interface PropsConfig {
  [prop: string]: unknown;

  // 样式控制
  'class'?: string;
  'style'?: StyleConfig;
  'appearance'?: string;

  // 带 name 的 slot
  'slot'?: string;
}

interface ActionConfig {
  action: string;
  option?: Record<string, unknown>;
}

interface StyleConfig {
  [key: string]: string | number;
}

// 含插值&作用域
type dataExtends = Record<string, unknown>;

// 静态值
type Data = Record<string, unknown>;
```

JSON 结构化协议 | 协议示例

```
{
  "ui": {
    "type": "score",
    "props": {
      "max": 5,
      "value": 3,
      "score": true
    }
  }
}

// -----
{
  "ui": {
    "type": "score",
    "props": {
      "max": 5,
      "value": "{{score}}",
      "score": true
    }
  }
}

{
  "data": {
    "score": 3
  }
}
```

无动态值

有动态插值

```
{
  "ui": {
    "type": "score",
    "props": {
      "max": 5,
      "value": "{{finalScore}}",
      "score": true
    }
  },
  "dataExtends": {
    "finalScore": "{{score + 1}}"
  }
}

{
  "data": {
    "score": 2
  }
}

// ----
{
  "type": "score",
  "for": "item, index in list",
  "props": {
    "max": 5,
    "value": "{{item}}",
    "score": true
  }
}

{
  "data": {
    "list": [1, 2, 3, 4, 5]
  }
}
```

有内置扩展数据

有动态值 + 逻辑



3.0 ★★☆☆☆

3.0 ★★☆☆☆

3.0 ★★☆☆☆

1.0 ★☆☆☆☆

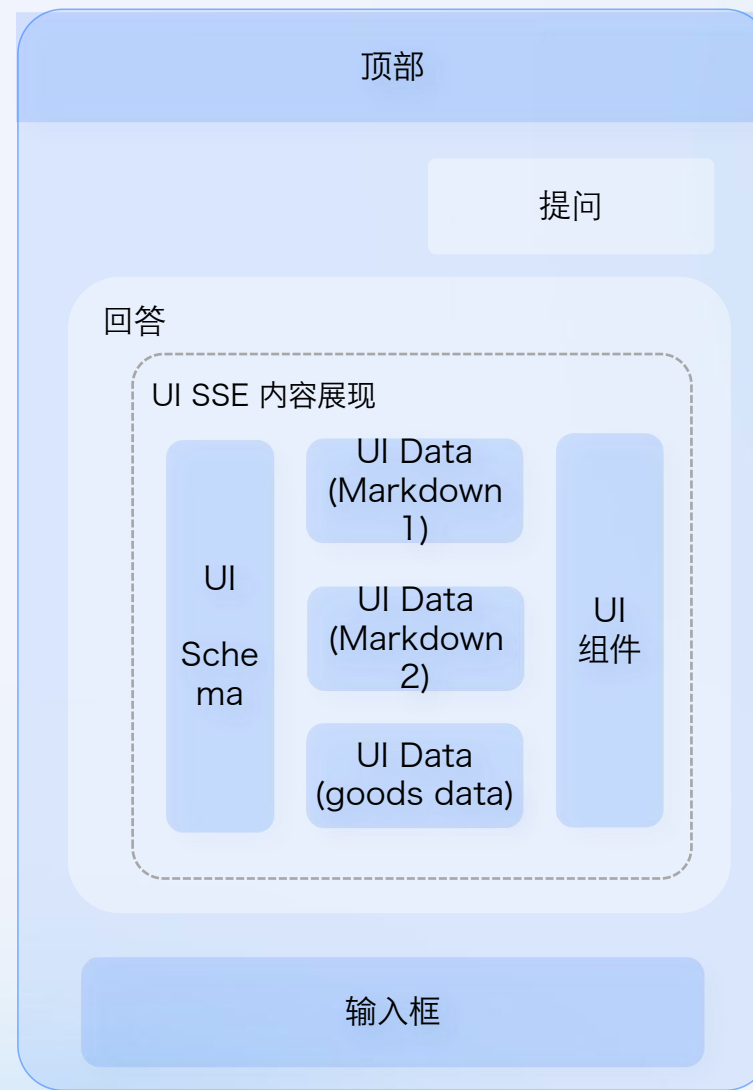
2.0 ★★☆☆☆

3.0 ★★☆☆☆

4.0 ★★☆☆☆

5.0 ★★☆☆☆

JSON 结构化协议 | 协议应用



AI 搜索应用中的 UI 工程实践 | 协议对比

| 协议 | Markdown 扩展协议 | JSON 结构化协议 |
|------|-----------------------|-----------------------|
| 协议定位 | 内容优先：文本增强 + Directive | 结构优先：Schema + Data |
| 结构模型 | 弱结构：内容 + 局部指令 | 树结构：组件嵌套 + 作用域 |
| 表达能力 | 中：文本为主 + 扩展组件 | 高：任意组件树 + 逻辑控制 + 交互行为 |
| 核心优势 | 生成成本低、流式天然友好 | 模板表达自然、声明式重渲染友好 |
| 适用场景 | LLM 流式输出、内容为主 + 轻交互 | 多行业结构化卡片、复杂布局 |

AI 搜索应用中的 UI 工程实践 | A2UI 协议

| Feature | v0.8.1 | v0.9 |
|----------------------|--|---|
| Philosophy | Structured Output / Function Calling | Prompt-First / In-Context Schema |
| Message Types | <code>beginRendering</code> , <code>surfaceUpdate</code> , ... | <code>createSurface</code> , <code>updateComponents</code> , ... |
| Surface Creation | Explicit <code>beginRendering</code> | Explicit <code>createSurface</code> |
| Component Type | Key-based wrapper (<code>{ "Text": ... }</code>) | Property-based discriminator (<code>"component": "Text"</code>) |
| Data Model Update | Array of Key-Value Pairs | Standard JSON Object |
| Data Binding | <code>dataBinding</code> / <code>literalString</code> | <code>path</code> / Native JSON types |
| Button Context | Array of Key-Value pairs | Standard JSON Object |
| Button Variant | Boolean (<code>primary: true</code>) | Enum (<code>variant: "primary"</code>) |
| Catalog | Separate component and function catalogs | Unified Catalog (<code>basic_catalog.json</code>) |
| Auxiliary Rules | N/A | <code>basic_catalog_rules.txt</code> |
| Validation | Basic Schema | Strict <code>ValidationFailed</code> feedback loop |
| Data Synchronization | Implicit | Explicit Client->Server data syncing (<code>sendDataModel</code>) |

A2UI

- v0.8.1 structured output
- **v0.9 Prompt first**
- 指令驱动: UI 操作流
- 扁平结构: adjacency list + ID 引用
- 显式状态更新、更偏状态驱动协议

- **v0.8.1** was designed to be generated by an LLM using Structured Output above all else, optimized for LLMs that support strict JSON mode or function calling (which is also a form of Structured Output). It relied on deep nesting and specific wrapper structures that were definable in the limited schema features but often confusing for an LLM to generate.
- **v0.9** is designed to be **embedded directly in an LLM's system prompt**. The schema is refactored to be more human-readable and "token-efficient" for the model to understand. It prioritizes patterns that LLMs naturally excel at (like standard JSON objects for maps) over strict structured output-friendly structures (like arrays of key-value pairs).

AI 搜索应用中的 UI 工程实践 | 整体架构设计



AI 搜索应用中的 UI 工程实践 | **Cosui** 对外开源



已在 Github Baidu 开源



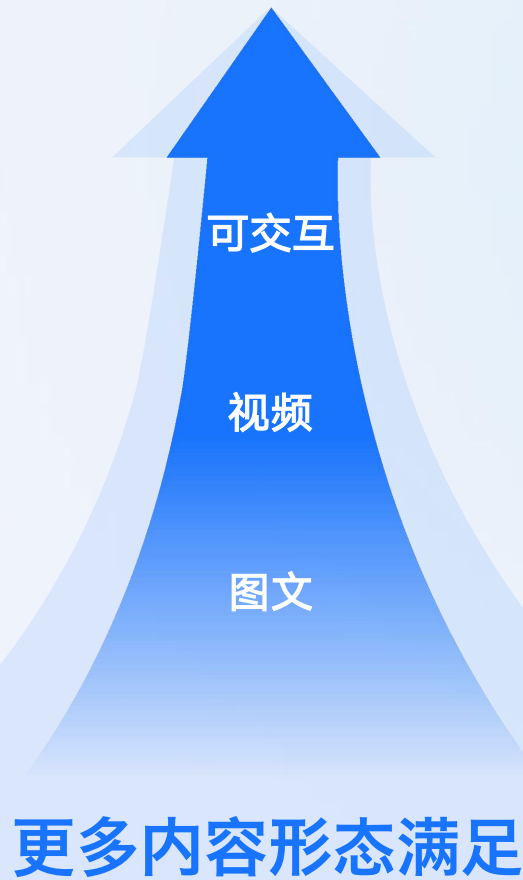
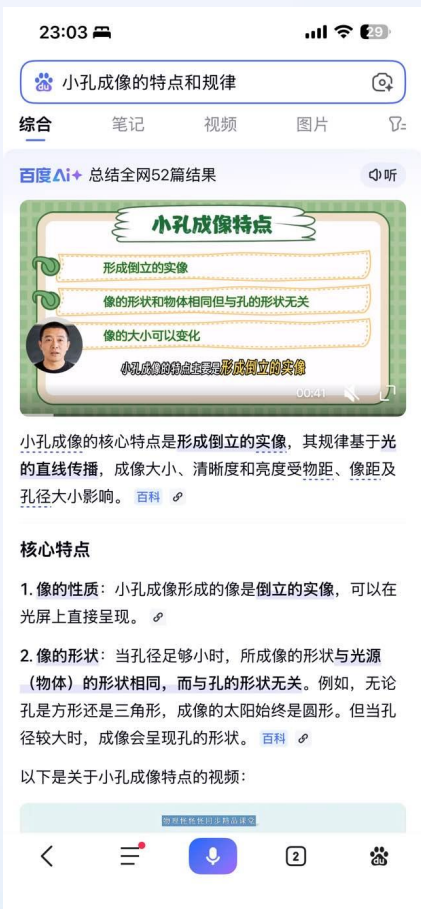
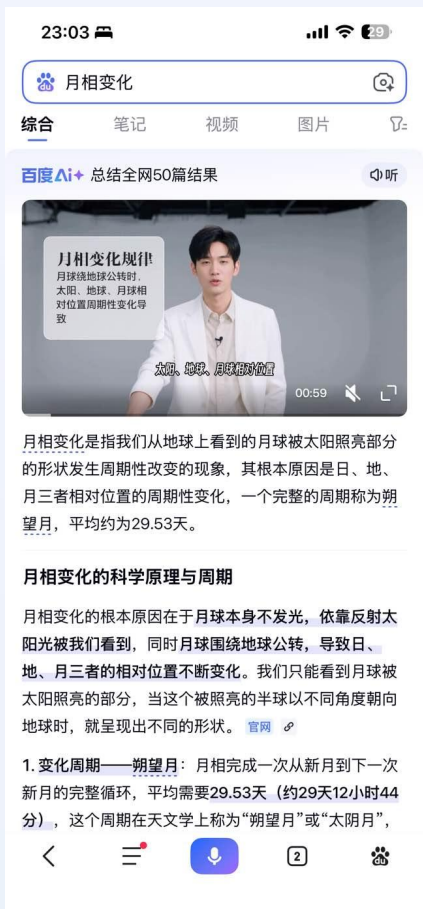
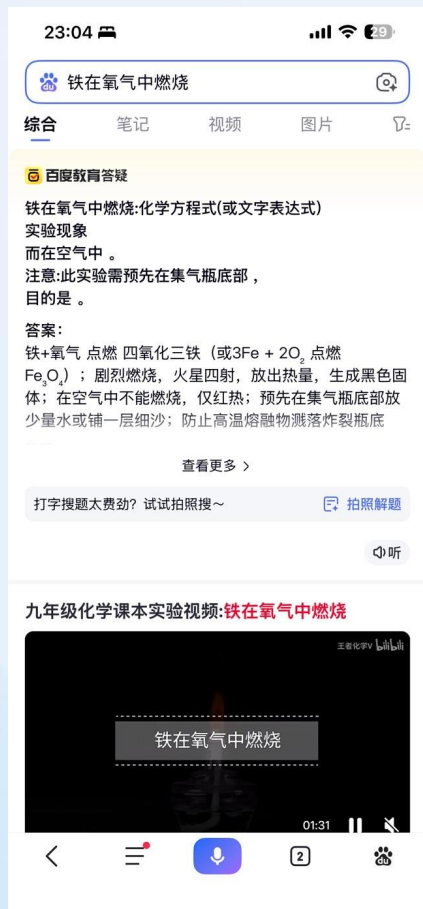
Github: <https://github.com/baidu/cosui>

官网: <https://baidu.github.io/cosui/>

03

生成式 UI 自动化产线的工程探索

生成式 UI 自动化产线的工程探索 | 探索思考



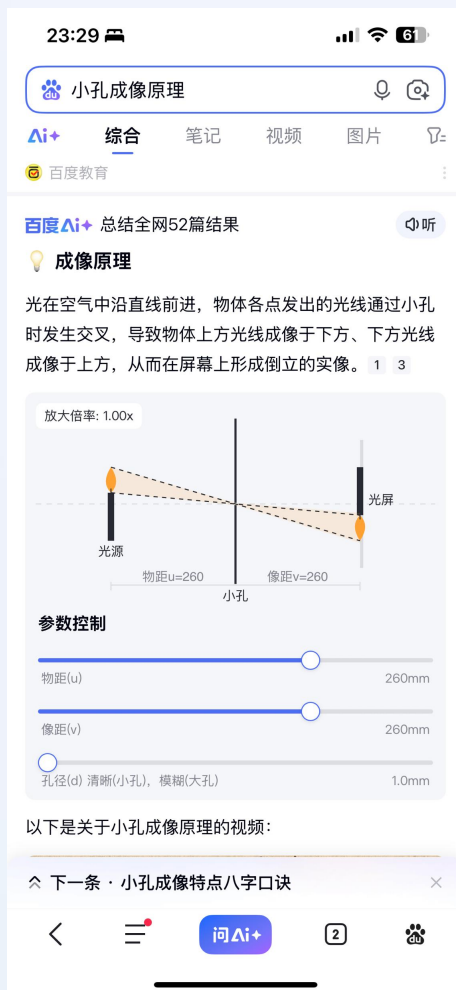
生成式 UI 自动化产线 | 快速验证

1 抽取 query & 提取需求

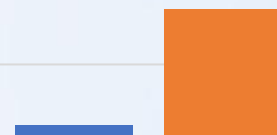
2 AI 工具雏型生产

3 设计规范对齐 & 人工微调

4 快速试验上线



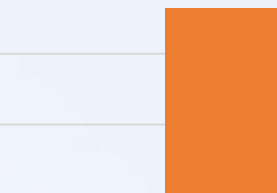
单 query 检索次数



检索次数

■ 上线前 ■ 上线后

单 query 点击量



点击量

■ 上线前 ■ 上线后

验证可行性

效果上线

效果收益

生成式 UI 自动化产线 | 量产需考虑的问题

体验增益

- 什么时候出？
- 出了是不是用户体验更好？

设计约束

- 产品下的设计规范
- 功能精细化的设计要求

研发约束

- 研发规范
- 运行隔离

AI 不稳定性

- 内容正确性问题
- 生产效果质量不稳定
- 生产速度不确定



生成式 UI 自动化产线 | 产品体验增益？

1 产品体验增益

什么场景适合出？

有快捷答案

- 今天天气怎么样？
- 长江有多长？

信息密度高或需要动手

- 长江经过哪些地区？
- 小孔成像原理是什么？

做什么做多少判断？

- 不能无限做、该做什么做多少？

生成式 UI 自动化产线 | 产品体验增益？

1 产品体验增益

什么场景适合出？

有快捷答案

- 今天天气怎么样？
- 长江有多长？

信息密度高或需要动手

- 长江经过哪些地区？
- 小孔成像原理是什么？

做什么做多少判断？

- 不能无限做、该做什么做多少？



解决：例行 query 评估

先验：指导工作方向

后验：指导工作过程&细粒度拆解
判断理想和实际之间的 Gap

覆盖天花板



Query 评估

当 $q=xxx$

是否能出？

出了是不是用户体验更好？



工具排产

覆盖 Query 抽象聚类

- 行业分类
- 预想实现方案
- 用户需求场景
- 交互形态

理想占比

实际占比

| | query总数量 | query总数量-去噪后 | query理想满足数量 | query理想对应组件数 | query实际满足数量 | query对应实际组件数 |
|---|----------|--------------|-------------|--------------|-------------|--------------|
| 1 | 8781 | 5093 | 1539 | 68 | 172 | 12 |
| 2 | | | | | | |

- 本批排产组件：68个
- 产线覆盖组件率：12 / (12+68) = 15%
- query：原始 8781 / 适合排产 1539 / 不适合排产 1977 / 已生产 172 / 去噪+去重+去空 5093

| 组件分类 | 排产原因 | 功能描述 | query 集 | 优先级 | 预估收益 |

生成式 UI 自动化产线 | 设计&研发约束问题?

2 设计约束

AI 自主发挥生成不符合产品下的设计要求

- **颜色问题**: 颜色不对、需要用主题色、暗黑下颜色不明显、日间要白色、夜间要黑色、A业务要蓝色、B业务要紫色、C业务要橙色 等等…
- **文字间距问题**: 文字不对齐、间距太小、太大、太粗、太细、太丑, 这里少1px, 那里加 2px 、等等…
- **功能布局问题**: 浮层在左边、功能多要2栏分布、功能少则独占一行、上下布局必须有标题、用户滑动操作要有震动 等等…

3 研发约束

- 代码要求: 日志打点要求、日夜间下不同处理要求、通信要求
- 性能要求: 代码体积限制, 避免无用冗长代码
- 运行环境要求: 多实例问题

生成式 UI 自动化产线 | 设计&研发约束问题?

2 设计约束

AI 自主发挥生成不符合产品下的设计要求

- **颜色问题**: 颜色不对、需要用主题色、暗黑下颜色不明显、日间要白色、夜间要黑色、A业务要蓝色、B业务要紫色、C业务要橙色 等等…
- **文字间距问题**: 文字不对齐、间距太小、太大、太粗、太细、太丑, 这里少1px, 那里加 2px、等等…
- **功能布局问题**: 浮层在左边、功能多要2栏分布、功能少则独占一行、上下布局必须有标题、用户滑动操作要有震动 等等…

解决: 规范对齐&规则沉淀、CLI & skill 工具校验评估

制定设计规范

明确 AI 自由发挥区、功能规范限制区



Design Token & 研发规则 & 组件沉淀

已有 Design token 规范输入 + AI 生产规范组件沉淀

CLI & skill 工具校验评估

- 确定性内容: 脚本处理、校验+自动修复
- 非确定性内容: 大模型+skill 处理、校验自动评估及修复
- 生产时 + 编译时 + 运行时处理

3 研发约束

- 代码要求: 日志打点要求、日夜间下不同处理要求、通信要求
- 性能要求: 代码体积限制, 避免无用冗长代码
- 运行环境要求: 多实例问题



生成式 UI 自动化产线 | AI 不稳定问题

4 AI 不稳定性



- **内容正确性**：已有规范，仍存生产错误、本身生产错误
- **生产质量不稳定**：已有规范，仍存非预期

解决：自动评估 + 例行评估、持续优化

生产时

- few shot + tool use
- 生产中进行自动校验修复

生产后

- 生产后自动评估
- 人工 + 专家评审
- 工具 + 人工干预

周期性评估

- 周期性评估效果，根据评估结果不断优化工程

生成式 UI 自动化产线 | AI 不稳定问题

4 AI 不稳定性

- **内容正确性**：已有规范，仍存生产错误、本身生产错误
- **生产质量不稳定**：已有规范，仍存非预期

解决： 自动评估 + 例行评估、持续优化

生产时

- few shot + tool use
- 生产中进行自动校验修复

生产后

- 生产后自动评估
- 人工 + 专家评审
- 工具 + 人工干预

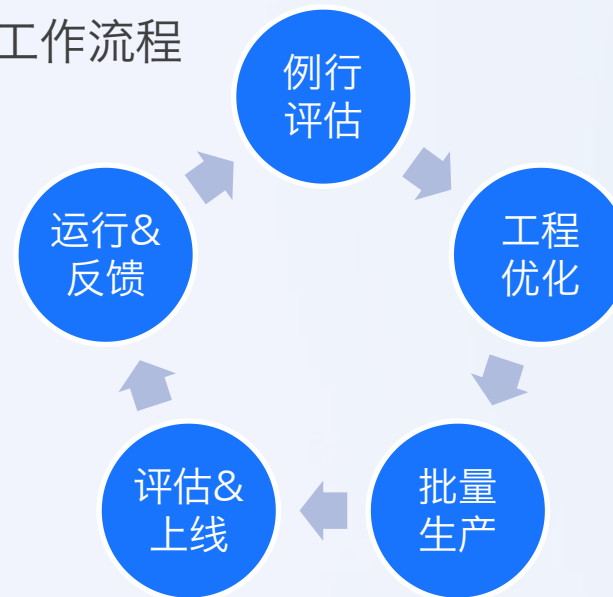
周期性评估

- 周期性评估效果，根据评估结果不断优化工程

- **生产速度不确定**

解决： 支持离线、在线生产两种模式

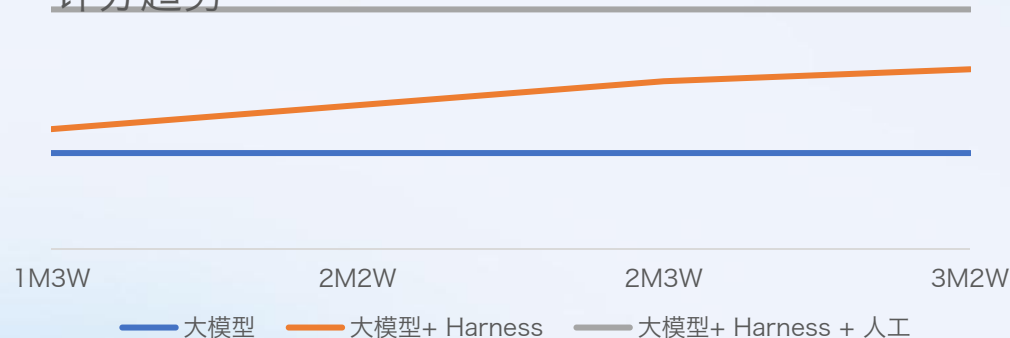
工作流程



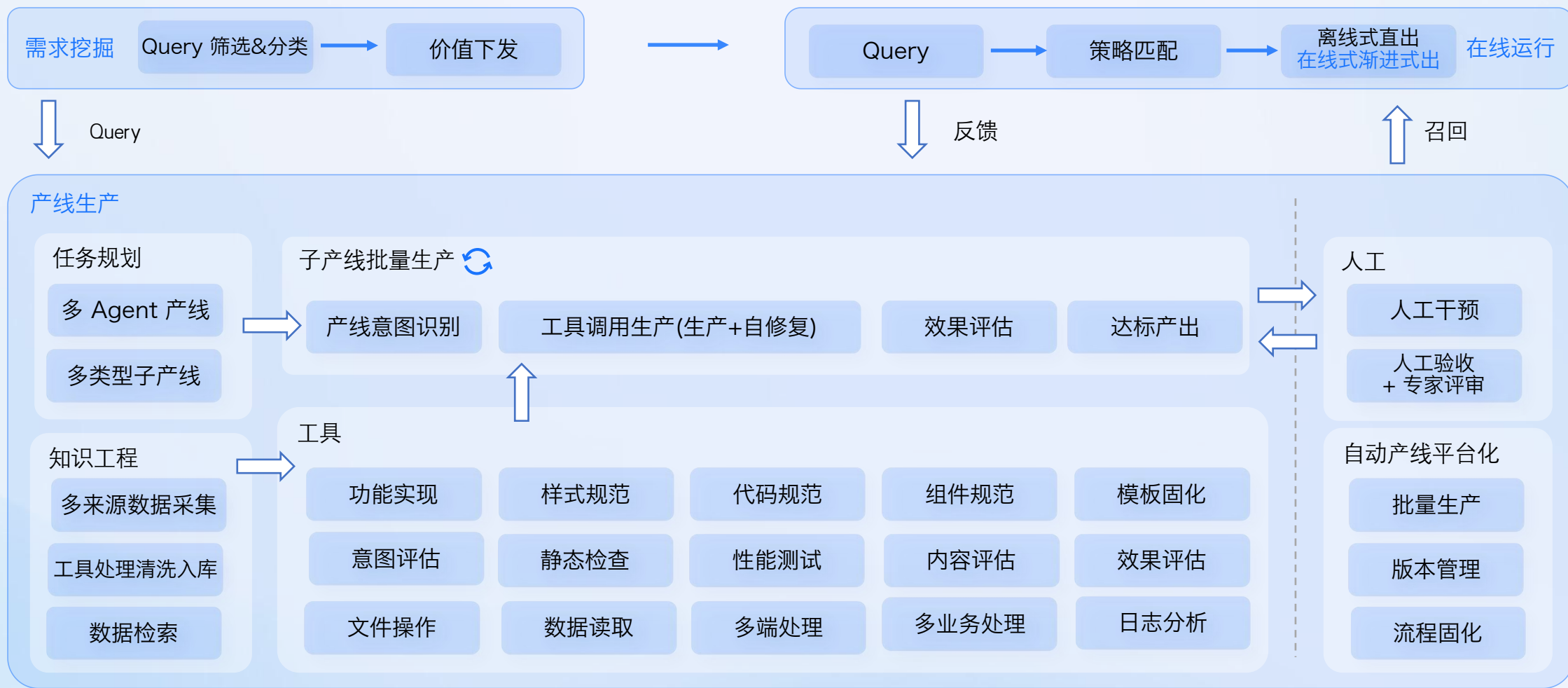
工作模式

- 效果先行验证
- 生产结果与规范收敛
- 工程实现与上线

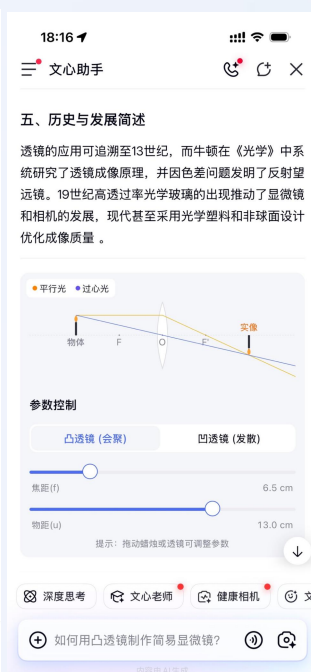
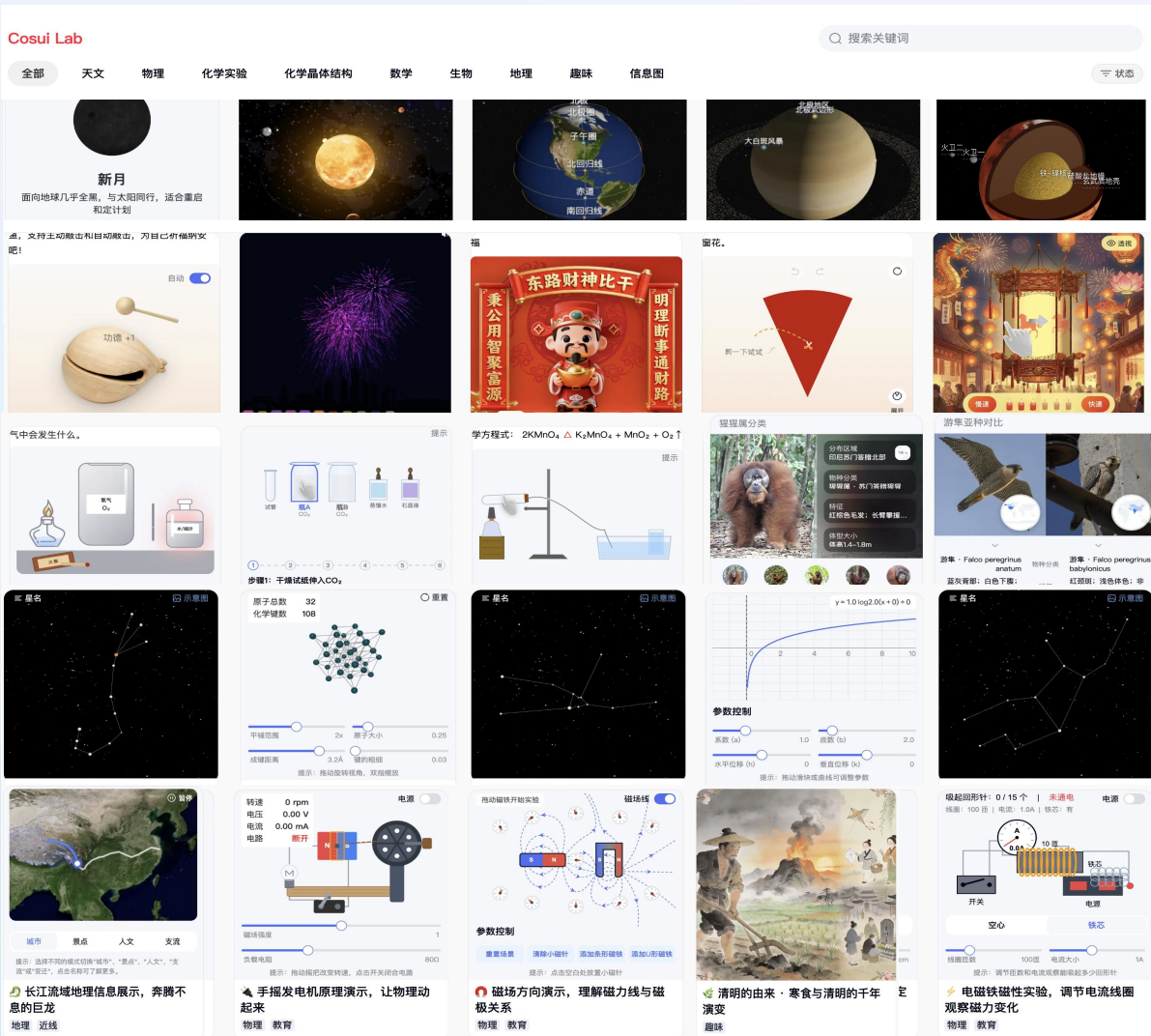
评分趋势



生成式 UI 自动化产线 | 产线工程



生成式 UI 自动化产线的工程探索 | 落地成果



5+
业务线接入

9+
学科类型

1000+
交互式组件

多端&多形态
亿级流量场景稳定运行

06

总结

总结

AI 搜索应用中的 UI 工程实践

- Markdown 扩展协议 & JSON 结构化协议的设计思路
- 协议落地中关键问题的解决方案
- 已在 GitHub baidu/cosui 开源

生成式 UI 自动化产线的工程探索

- 从快速验证到批量化生产的路径
- 平台化产线的工程架构
- 产品、设计、研发及 AI 不稳定性的工程兜底

THANKS

大模型正在重新定义软件

Large Language Model Is Redefining The
Software



AiCon

全球人工智能开发与应用大会

上海站

探索 AI 应用边界

Explore the limits of AI applications

2026 年 6 月 26-27 日 / 上海张江科学会堂



参会咨询

专题：世界模型与多模态智能突破

专题出品人：朱政

极佳视界 / 联合创始人 & 首席科学家



专题：Agent 系统架构与工程化实践

专题出品人：鲁浩楠

OPPO / 大模型算法负责人



专题：AI 原生数据工程

专题出品人：王彦辉

火山引擎 / 数智平台产品总监



专题：Agent 安全、评测与可信治理

专题出品人：马传雷（岳立）

支付宝 / 业务风险技术部负责人



专题：Agent 数据、记忆与运行时基础设施

专题出品人：邓亚峰

EverMind / CEO



专题：企业级研发体系的重构

专题出品人：沈浪

快手 / 研发效能负责人

